

Apply filters to SQL queries

Project description

In this project, I used SQL queries to investigate security-related events and identify employee devices that require security updates. I applied different SQL filtering techniques, including **WHERE**, **AND**, **OR**, **NOT**, and **LIKE**, to retrieve specific information from the `log_in_attempts` and `employees` tables. These queries helped narrow down large amounts of data and supported the investigation of suspicious login activity and system maintenance.

Retrieve after hours failed login attempts

SQL Query

```
SELECT *  
FROM log_in_attempts  
WHERE login_time > '18:00'  
AND success = FALSE;
```

Explanation

This query retrieves all login attempts that occurred after **6:00 PM (18:00)** and were unsuccessful. The **WHERE** clause filters records based on the specified conditions. The **AND** operator ensures that both conditions must be true: the login attempt must have occurred after business hours and the login must have failed (**FALSE**). This helps identify potentially suspicious after-hours login activity.

Retrieve login attempts on specific dates

SQL Query

```
SELECT *  
FROM log_in_attempts
```

```
WHERE login_date = '2022-05-09'  
OR login_date = '2022-05-08';
```

Explanation

This query retrieves all login attempts that occurred on **May 8, 2022** or **May 9, 2022**. The **OR** operator allows records from either date to be included in the results. This query is useful for investigating login activity around the time of a reported security incident.

Retrieve login attempts outside of Mexico

SQL Query

```
SELECT *  
FROM log_in_attempts  
WHERE NOT country LIKE 'MEX%';
```

Explanation

This query retrieves all login attempts that did **not** originate from Mexico. The **LIKE** operator with the **%** wildcard matches values such as **MEX** and **MEXICO**. The **NOT** operator excludes those records, returning only login attempts from other countries. This helps investigators focus on activity outside the expected location.

Retrieve employees in Marketing

SQL Query

```
SELECT *  
FROM employees  
WHERE department = 'Marketing'  
AND office LIKE 'East%';
```

Explanation

This query retrieves employees who work in the **Marketing** department and are located in the **East** building. The **LIKE** 'East%' pattern matches office locations such as **East-170** and **East-320**. The **AND** operator ensures that employees satisfy both conditions before they are included in the results.

Retrieve employees in Finance or Sales

SQL Query

```
SELECT *  
FROM employees  
WHERE department = 'Finance'  
OR department = 'Sales';
```

Explanation

This query retrieves employees who belong to either the **Finance** department or the **Sales** department. The **OR** operator allows records from either department to appear in the results. These employees were identified because they require a different security update.

Retrieve all employees not in IT

SQL Query

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

Explanation

This query retrieves all employees except those in the **Information Technology** department. The **NOT** operator excludes IT employees because their devices

have already received the required security update. The resulting list identifies employees whose machines still need to be updated.

Summary

In this project, I used SQL filtering techniques to investigate login activity and identify employee devices requiring security updates. I applied filters using **dates, times, AND, OR, NOT, and LIKE** to retrieve only the relevant records from the database. These queries demonstrate how SQL can be used in cybersecurity to analyze security events, investigate suspicious activity, and efficiently locate systems that require administrative action.